
```

% assignment 3.1
function ass3_1
% load reservoir data X(11x1), Y(21x1), Z(21x11)
[x, y, Z] = reservoir_data;
% create a mesh of 21 x 11 matrices for X, Y locations
[X,Y]=meshgrid(x,y);
surf(X,Y,Z);
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% compute volume using simpson's rule
f=@(x)depth_func(X,Y,Z,x);
a=[-100,0];
b=[100,400];
n=10;
vol=gsimp(f,a,b,n);
fprintf('The volume of reservoir using simpson rule is %g m^3\n',vol);
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% compute force using simpson's rule
a=-100;
b=100;
f=@(x)force_func(X,Y,Z,x);
n=20;
F=gsimp(f,a,b,n);
fprintf('The force acting on the dam is %g KN\n',F/1000);
return

% depth function for simpson integration
% input x,y,z as row vectors in x(m)
% use notation x = x(1); y=x(2); z=x(3) etc...
function [z] = depth_func(X,Y,Z,x)
% note that x,y are contained in x(1), x(2) respectively
XI=x(1); YI=x(2);
% interpolate to find the depth z at XI and YI
z=-interp2(X,Y,Z,XI,YI);
return

% force function for simpson integration
% input x,y,z as row vectors in x(m)
% use notation x = x(1); y=x(2); z=x(3) etc...
function [force] = force_func(X,Y,Z,x)
rho=1000; g=9.81;
% interpolate to find depth z at y = 0
z0=interp1(X(1,:),Z(1,:),x(1));
force=0.5*rho*g*z0^2;
return

% reservoir data is included in function
function [X, Y, Z] = reservoir_data
X = [-100:20:100]';
Y = [0:20:400]';

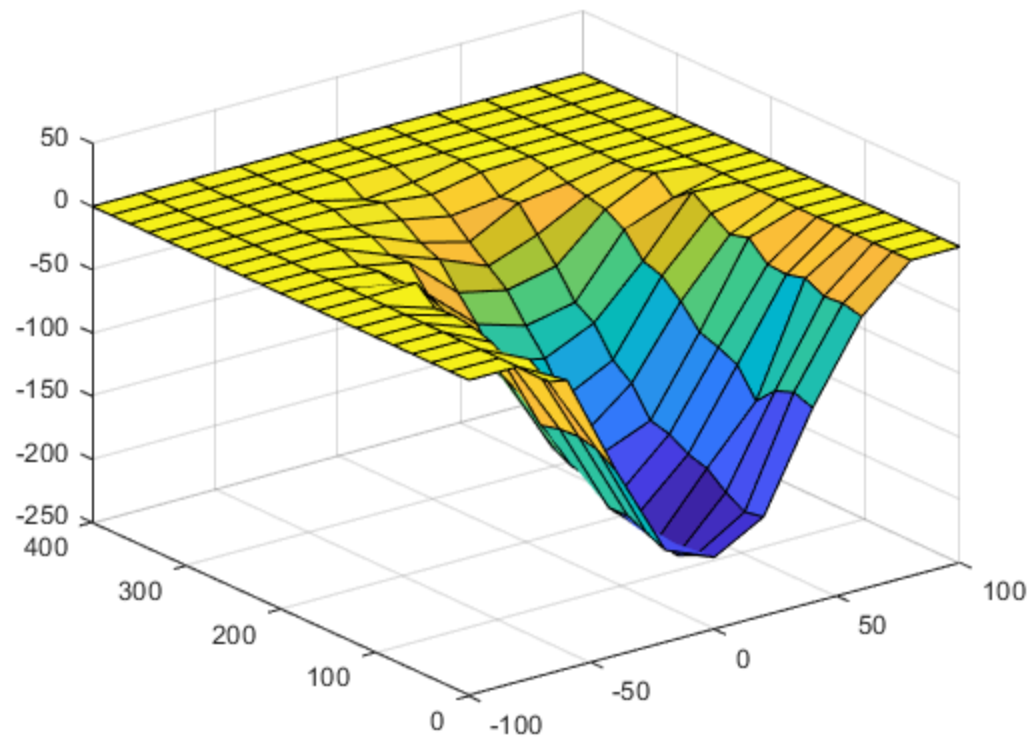
```

```

Z = [
0 0 0 0 0 0 0 ...
0 0 0 0 0 0 0 ...
0 0 0 0 0 0 0
0 0 0 0 0 0 0 ...
0 0 0 0 0 0 0 ...
0 0 0 0 0 0 0
-21.165 -29.4643 -18.2436 -28.6525 -18.6106 -13.7847 -11.2466 ...
-12.3863 0.91977 -7.67067 -3.74114 -6.43724 0 0 ...
0 0 0 0 0 0 0
-103.725 -93.787 -90.063 -91.0594 -95.0413 -78.5611 -70.1245 ...
-59.4917 -53.0261 -40.778 -32.5335 -14.121 -15.1776 -7.11059 ...
-5.95325 0 0 0 0 0 0
-176.85 -170.092 -163.33 -163.842 -147.099 -144.317 -136.609 ...
-111.888 -105.932 -89.8775 -66.6214 -48.5372 -29.0193 -22.6001 ...
-11.538 -4.70103 -9.3584 0 0 0 0
-192.872 -198.179 -205.256 -198.799 -185.208 -175.84 -152.348 ...
-144.676 -122.149 -97.496 -78.41 -57.1481 -44.9376 -32.6432 ...
-19.1221 -9.0154 -6.05826 0 0 0 0
-171.567 -174.491 -165.773 -153.745 -149.361 -140.429 -136.499 ...
-113.841 -102.345 -79.8951 -70.4826 -48.5335 -30.6694 -12.4041 ...
-14.4333 -1.63678 0.818634 0 0 0 0
-95.8321 -90.4387 -96.1188 -111.043 -89.2533 -82.4199 -77.0819 ...
-59.6812 -46.2358 -41.3343 -23.5262 -14.3117 -10.568 -7.07419 ...
-2.19095 0 0 0 0 0 0
-34.217 -30.5423 -31.8591 -24.236 -28.236 -27.1394 -12.9377 ...
-17.2499 -7.33462 2.72072 -8.77947 4.19974 0 0 ...
0 0 0 0 0 0 0
0 0 0 0 0 0 0 ...
0 0 0 0 0 0 0 ...
0 0 0 0 0 0 0
0 0 0 0 0 0 0 ...
0 0 0 0 0 0 0 ...
0 0 0 0 0 0 0
]';
return

```

The volume of reservoir using simpson rule is 2.72745e+06 m³
The force acting on the dam is 1.13274e+07 KN



Published with MATLAB® R2020a

PROBLEM 2

```
function ass3_2
f=@fxyz;
a={0,0,0};
b={8,@ymax,@zmax};
n=50;
% simpson
mass=feval(@gsimp,f,a,b,n);
fprintf('The mass of the region using simpson is %g\n',mass);
% integral3
mass=integral3(@(x,y,z) (exp(-(x.^2+y.^2+z.^2)./sqrt(1+z.^3))),0,8,0,...
    @(x) (2*x),0,@(x,y) (x+2*y));
fprintf('The mass of the region using integral3 is %g\n',mass);
return

% function
function [f] = fxyz(x)
f = exp(-(x(1)^2+x(2)^2+x(3)^2)/sqrt(1+x(3)^3));
return

function [f] = ymax(x)
f=2*x(1);
return

function [f] = zmax(x)
f=x(1)+2*x(2);
return

% multi-dimensional simpson main calling function
% goes from outer to inner (i.e., x to z).
function [I] = gsimp(f,a,b,n)
m=length(a); x=zeros(1,m); k=1;
% convert limits to cell array if numbers are given
if (isnumeric(a))
    a=num2cell(a);
end
if (isnumeric(b))
    b=num2cell(b);
end
% call recursive integration function
I=ksimp(f,x,a,b,n,k);
return

% recursive simpson function
function [I] = ksimp(f,x,a,b,n,k)
I=0; m=length(a);
if isnumeric(a{k})
    ak=a{k};
else
    ak=feval(a{k},x);
end
if isnumeric(b{k})
    bk=b{k};
else
    bk=feval(b{k},x);
end
h = (bk-ak)/n;
```

```

for i=0:n
    x(k)=ak+i*h;
    if (k == m)
        if (i == 0 || i == n)
            I=I+feval(f,x);
        elseif (mod(i,2) == 1)
            I=I+4*feval(f,x);
        elseif (mod(i,2) == 0)
            I=I+2*feval(f,x);
        end
    else
        if (i == 0 || i == n)
            I=I+feval(@ksimp,f,x,a,b,n,k+1);
        elseif (mod(i,2) == 1)
            I=I+4*feval(@ksimp,f,x,a,b,n,k+1);
        elseif (mod(i,2) == 0)
            I=I+2*feval(@ksimp,f,x,a,b,n,k+1);
        end
    end
end
I=I*h/3;
return

```

The mass of the region using simpson is 4.53361
The mass of the region using integral3 is 4.53335

Problem 3

assignment 3.3 Gauss quadrature examples

```
function ass3_3
% problem 3.2(a)
% evaluate single integral using Gauss quadrature
f=@ln1dx;
% need to input the limits as characters as we want to
% be able to handle function names as limits as well
a=0; % lower limit of integral
b=1; % upper limit of integral
n=5; % number of gauss points
m=1; % number of dimensions = 3
I3n=feval(@ggauss,f,a,b,n);
fprintf('The Gauss Quardrature value of I1 = %g\n',I3n);
% analytical evaluation of triple integral
I3a=sqrt(pi)/2;
fprintf('The analytical value of I1 = %g\n',I3a);
error=100*abs(I3n-I3a)/abs(I3a);
fprintf('Percentage error of numerical solution %g %%\n\n',error);
%
%
% problem 3.2(b)
% evaluate single integral using Gauss quadrature
f=@xlnsinx;
% need to input the limits as characters as we want to
% be able to handle function names as limits as well
a=0; % lower limit of integral
b=pi; % upper limit of integral
n=5; % number of gauss points
I3n=feval(@ggauss,f,a,b,n);
fprintf('The Gauss Quardrature value of I2 = %g\n',I3n);
% analytical evaluation of triple integral
I3a=-0.5*pi^2*log(2);
fprintf('The analytical value of I2 = %g\n',I3a);
error=100*abs(I3n-I3a)/abs(I3a);
fprintf('Percentage error of numerical solution %g %%\n\n',error);
%
% problem 3.2(c)
% evaluate single integral using Gauss quadrature
f=@clnx;
% need to input the limits as characters as we want to
% be able to handle function names as limits as well
a=0; % lower limit of integral
b=1; % upper limit of integral
n=5; % number of gauss points
I3n=feval(@ggauss,f,a,b,n);
fprintf('The Gauss Quardrature value of I3 = %g\n',I3n);
% analytical evaluation of triple integral
I3a=pi;
fprintf('The analytical value of I3 = %g\n',I3a);
error=100*abs(I3n-I3a)/abs(I3a);
fprintf('Percentage error of numerical solution %g %%\n\n',error);
return

function [fx] = ln1dx(x)
fx=sqrt(log(1/x(1)));
```

```

return

function [fx] = xlnsinx(x)
fx=x*log(sin(x));
return

function [fx] = clnx(x)
fx=1/(x*(1-log(x))*sqrt(log(1/x)));
return

% general gauss quadrature for an integral of m dimensions
function [I] = ggauss(f,a,b,n)
m = length(a);
if (n == 1)
    zeta(1)=0;
    w(1)=2;
elseif (n == 2)
    zeta(1)=1/sqrt(3);
    zeta(2)=-zeta(1);
    w(1)=1;
    w(2)=1;
elseif (n == 3)
    zeta(1)=0;
    zeta(2)=sqrt(3/5);
    zeta(3)=-zeta(2);
    w(1)=8/9;
    w(2)=5/9;
    w(3)=w(2);
elseif (n == 4)
    zeta(1)=sqrt((3-2*sqrt(6/5))/7);
    zeta(2)=-zeta(1);
    zeta(3)=sqrt((3+2*sqrt(6/5))/7);
    zeta(4)=-zeta(3);
    w(1)=0.5+1/(6*sqrt(6/5));
    w(2)=w(1);
    w(3)=0.5-1/(6*sqrt(6/5));
    w(4)=w(3);
elseif (n == 5)
    zeta(1)=0;
    zeta(2)=sqrt(5-4*sqrt(5/14))/3;
    zeta(3)=-zeta(2);
    zeta(4)=sqrt(5+4*sqrt(5/14))/3;
    zeta(5)=-zeta(4);
    w(1)=128/225;
    w(2)=161/450+13/(180*sqrt(5/14));
    w(3)=w(2);
    w(4)=161/450-13/(180*sqrt(5/14));
    w(5)=w(4);
elseif (n == 6)
    w(1)=0.3607615730481386; zeta(1)=0.6612093864662645;
    w(2)=0.3607615730481386; zeta(2)=-0.6612093864662645;
    w(3)=0.4679139345726910; zeta(3)=-0.2386191860831969;
    w(4)=0.4679139345726910; zeta(4)=0.2386191860831969;
    w(5)=0.1713244923791704; zeta(5)=-0.9324695142031521;
    w(6)=0.1713244923791704; zeta(6)=0.9324695142031521;
else
    disp('This value of n not implemented!')
    return
end
m=length(a); x=zeros(1,m); k=1;

```

```

% convert limits to cell array if numbers are given
if (isnumeric(a))
    a=num2cell(a);
end
if (isnumeric(b))
    b=num2cell(b);
end
% in the beginning k=1
I=feval(@kgauss,f,x,zeta,w,a,b,k,n);
return

% this function is recursively called to evaluate a multiple
% integral of dimension 'm' using gauss quadrature
% it is assumed that the outermost integral is x(1)
% for example in a triple integral, x=x(1), y=x(2), z=x(3)
% int(int(int(f(x,y,z)*dx)dy)dz
function [Ik] = kgauss(f,x,zeta,w,a,b,k,n)
Ik=0; m=length(a);
if isnumeric(a{k})
    ak=a{k};
else
    ak=feval(a{k},x);
end
if isnumeric(b{k})
    bk=b{k};
else
    bk=feval(b{k},x);
end
for i=1:n
    x(k)=0.5*((bk-ak)*zeta(i)+(bk+ak));
    if (k == m)
        Ik=Ik+w(i)*feval(f,x);
    else
        Ik=Ik+w(i)*feval(@kgauss,f,x,zeta,w,a,b,k+1,n);
    end
end
Ik=Ik*(bk-ak)/2;
return

```

The Gauss Quadrature value of I1 = 0.882365
 The analytical value of I1 = 0.886227
 Percentage error of numerical solution 0.435815 %

The Gauss Quadrature value of I2 = -3.21329
 The analytical value of I2 = -3.42054
 Percentage error of numerical solution 6.05903 %

The Gauss Quadrature value of I3 = 2.12865
 The analytical value of I3 = 3.14159
 Percentage error of numerical solution 32.2428 %


```

% volume of an ellipsoid
function ass3_4_i
f=@ellipse;
A=4; B=2; C=1;
a = {-A,@(x)ymin(x,A,B,C),@(x)zmin(x,A,B,C)}; % xmin, ymin, zmin
b = {A,@(x)ymax(x,A,B,C),@(x)zmax(x,A,B,C)}; % xmax, ymax, zmax
n=20;
I_simp = real(gsimp(f,a,b,n))
% I_integral3 = integral3(f,-4,4,@ymin,@ymax,@zmin,@zmax)
syms x y z a b c;
S=((int(int(int(1,x,...
    -a*sqrt(1-(z/c)^2-(y/b)^2),a*sqrt(1-(z/c)^2-(y/b)^2)),...
    y,-b*sqrt(1-(z/c)^2),b*sqrt(1-(z/c)^2)),...
    z,-c,c)));
a=4; b=2; c=1;
I_sym = double(subs(S))
I_anal = 4*pi*A*B*C/3
return

function [f] = ellipse(x)
f = 1;
return

function [f] = ymin(x,A,B,C)
X=x(1);
f = -B*sqrt(1-(X/A).^2);
return

function [f] = ymax(x,A,B,C)
X=x(1);
f = A*sqrt(1-(X/A).^2);
return

function [f] = zmin(x,A,B,C)
X=x(1); Y=x(2);
f = -C*sqrt(1-(Y/B).^2-(X/A).^2);
return

function [f] = zmax(x,A,B,C)
X=x(1); Y=x(2);
f = C*sqrt(1-(Y/B).^2-(X/A).^2);
return

```

```

I_simp =

    33.594

```

```

I_sym =

    33.51

```

```

I_anal =

    33.51

```



```

% mass of contaminant in truncated ellipsoid
function ass3_4_ii
a = {-2,@ymin,@xmin};
b = {-1.1,@ymax,@xmax};
n=20;
mass_of_contaminant = real(gsimp(@ellipsoid_mass,a,b,n))
% clear;
% syms x y z f;
% I_anal_symbolic=double(int(int(int(0.1/sqrt(1+z^2),x,-10*sqrt(1-(z/2)^2-(y/4)^2),10*sqrt(1-
(z/2)^2-(y/4)^2)),...
%      y,-4*sqrt(1-(z/2)^2),4*sqrt(1-(z/2)^2)),z,-2,-1.25))
return

function [f] = ellipsoid_mass(x)
z=x(1); c0=0.1;
f = c0/sqrt(1+z^2);
return

function [f] = xmin(x)
y=x(2); z=x(1);
a=10; b=4; c=2;
f = -a*sqrt(1-(y/b)^2-(z/c)^2);
return

function [f] = xmax(x)
y=x(2); z=x(1);
a=10; b=4; c=2;
f = a*sqrt(1-(y/b)^2-(z/c)^2);
return

function [f] = ymin(x)
z=x(1);
a=10; b=4; c=2;
f = -b*sqrt(1-(z/c)^2);
return

function [f] = ymax(x)
z=x(1);
a=10; b=4; c=2;
f = b*sqrt(1-(z/c)^2);
return

```

```
mass_of_contaminant =
```

```
2.5059
```

```

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% mass in tank example %%%%%%%%%
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
function ass3_5_i
f=@tank_conc;
a=[0,0];
b=[2,3];
n=10;
M=feval(@gsimp,f,a,b,n);
fprintf('Mass in the tank is %g Kg\n',M);
return
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
% functions to be evaluated follows
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
function [f]=tank_conc(x)
% use notation x = x(1); y=x(2); z=x(3) etc...
C0=0.1; r=x(1); z=x(2);
f=C0*2*pi*r^3/sqrt(z^2+1);
return

```

Mass in the tank is 4.57023 Kg

Problem 5(ii)

Tank problem: Variable height

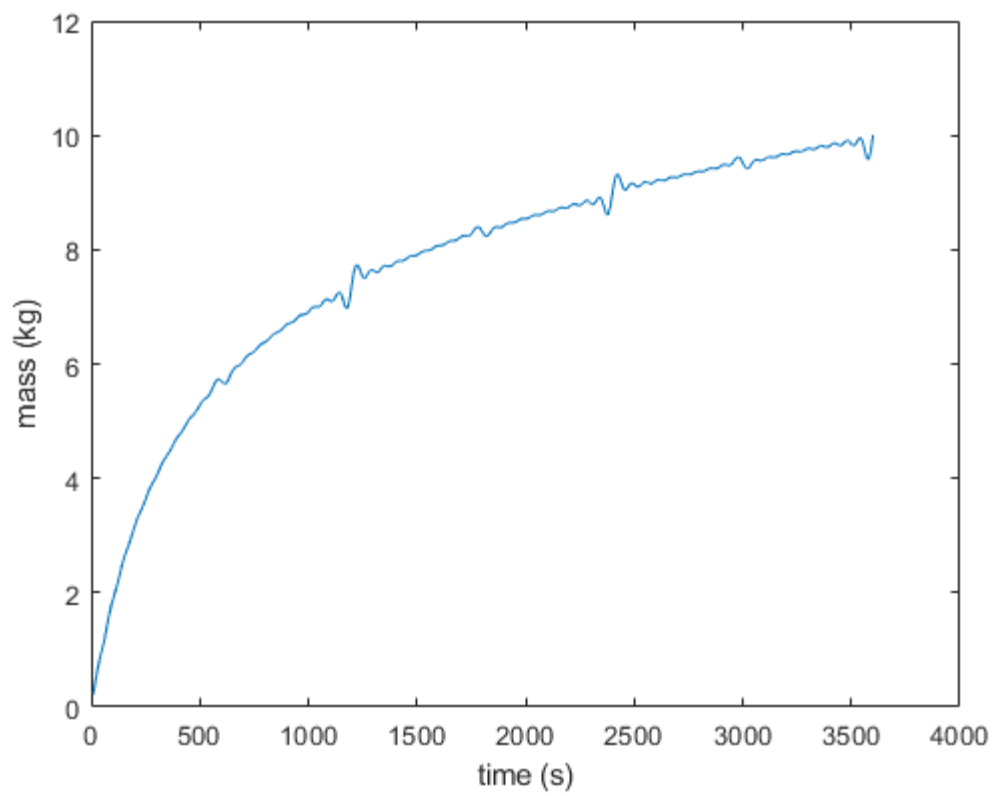
```
function ass3_5_ii
f=@tank_conc;
n=10;
T=3600; dt=10; nt=T/dt;
for i=1:nt
    t(i)=i*dt;
    zmax = tank_height(t(i));
    a={0,0};
    b={2,zmax};
    M(i)=feval(@gsimp,f,a,b,n);
end
plot(t,M);
xlabel('time (s)');
ylabel('mass (kg)');
fprintf('Mass in the tank at 1 hour is %g Kg\n',M(nt));
hold off;
return
%

function [f]=tank_conc(x)
% use notation x = x(1); y=x(2); z=x(3) etc...
C0=0.1; r=x(1); z=x(2);
f=C0*2*pi*r^3/sqrt(z^2+1);
return

function [h] = tank_height(T)
R=2; A=pi*R^2; n=100;
h=gsimp(@qfunc,0,T,n)/A;
return

function [q] = qfunc(t)
Q0=0.1; omega=pi/6;
q=Q0*(1+sin(omega*t));
return
```

Mass in the tank at 1 hour is 10.0064 Kg



Problem 5(iii)

Tank problem: Variable height and decaying concentration

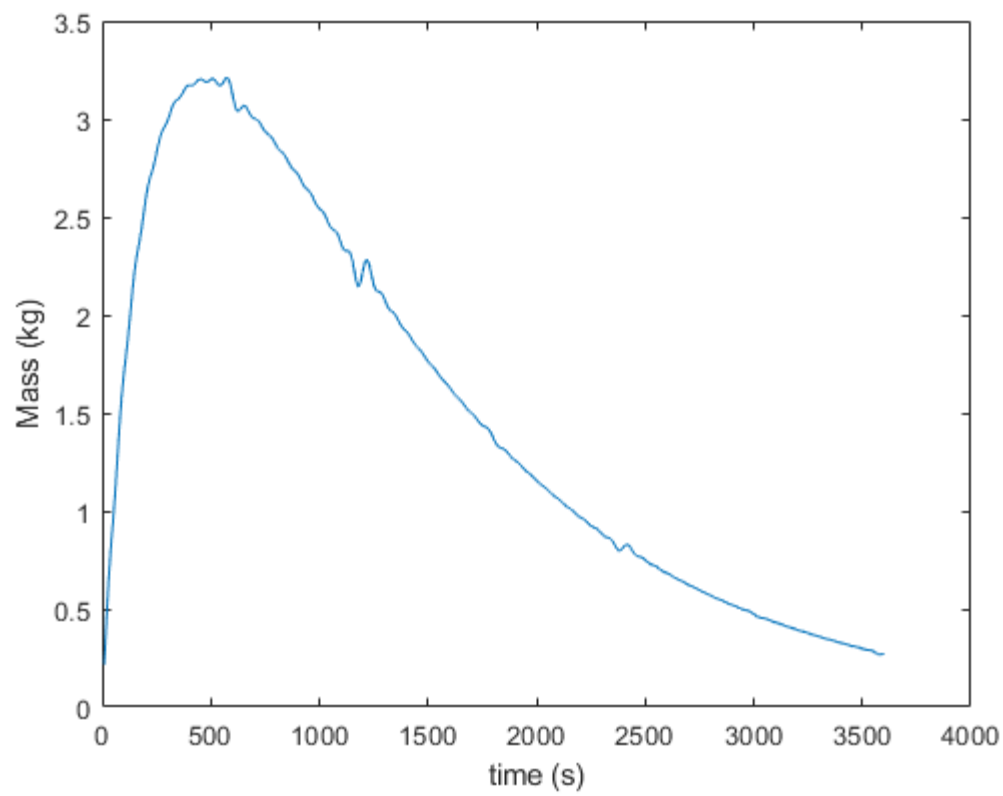
```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
function ass3_5_iii_a
n=10;
T=3600; dt=10; nt=T/dt;
for i=1:nt
    t(i)=i*dt;
    zmax = tank_height(t(i));
    a={0,0};
    b={2,zmax};
    f=@(x) tank_conc(x,t(i));
    M(i)=gsimp(f,a,b,n);
end
figure(1);
plot(t,M);
xlabel('time (s)');
ylabel('Mass (kg)');
hold off;
return

% functions to be evaluated as follows

function [f]=tank_conc(x,t)
% use notation x = x(1); y=x(2); z=x(3) etc...
C0=0.1; r=x(1); z=x(2); k=0.001;
f=C0*2*pi*r^3*exp(-k*t)/sqrt(z^2+1);
return

function [h] = tank_height(T)
R=2; A=pi*R^2; n=100;
h=gsimp(@qfunc,0,T,n)/A;
return

function [q] = qfunc(t)
Q0=0.1; omega=pi/6;
q=Q0*(1+sin(omega*t));
return
```



Problem 5(iib)

Tank problem: Variable height and decaying concentration

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
function ass3_5_iii_b
f=@tank_conc; T=3600;
a={0,0,0};
b={T,2,@tank_height};
n=10;
MT=gsimp(f,a,b,n);
fprintf('Average mass in the tank through 1 hour is %g Kg\n',MT/T);
return
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

% functions to be evaluated follows
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
function [f]=tank_conc(x)
% use notation x = x(1); y=x(2); z=x(3) etc...
C0=0.1; t=x(1); r=x(2); z=x(3); k=0.001;
f=C0*2*pi*r^3*exp(-k*t)/sqrt(z^2+1);
return

function [h] = tank_height(x)
R=2; A=pi*R^2; n=100; T=x(1);
h=gsimp(@qfunc,0,T,n)/A;
return

function [q] = qfunc(t)
Q0=0.1; omega=pi/6;
q=Q0*(1+sin(omega*t));
return
```

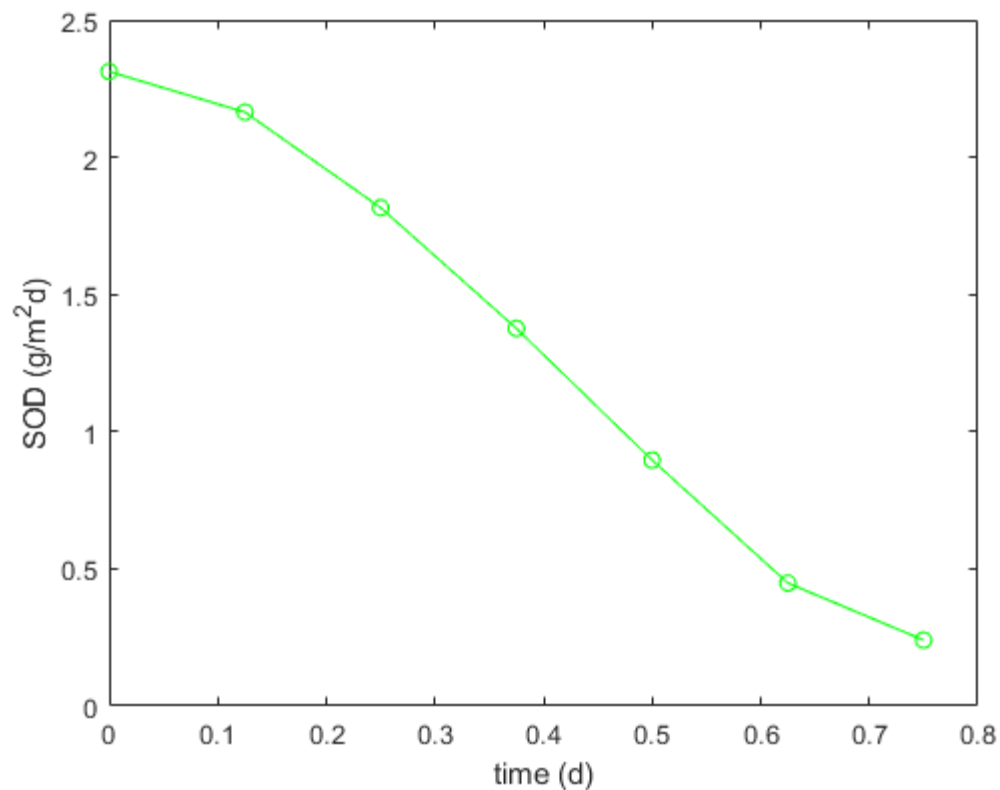
Average mass in the tank through 1 hour is 1.50381 Kg

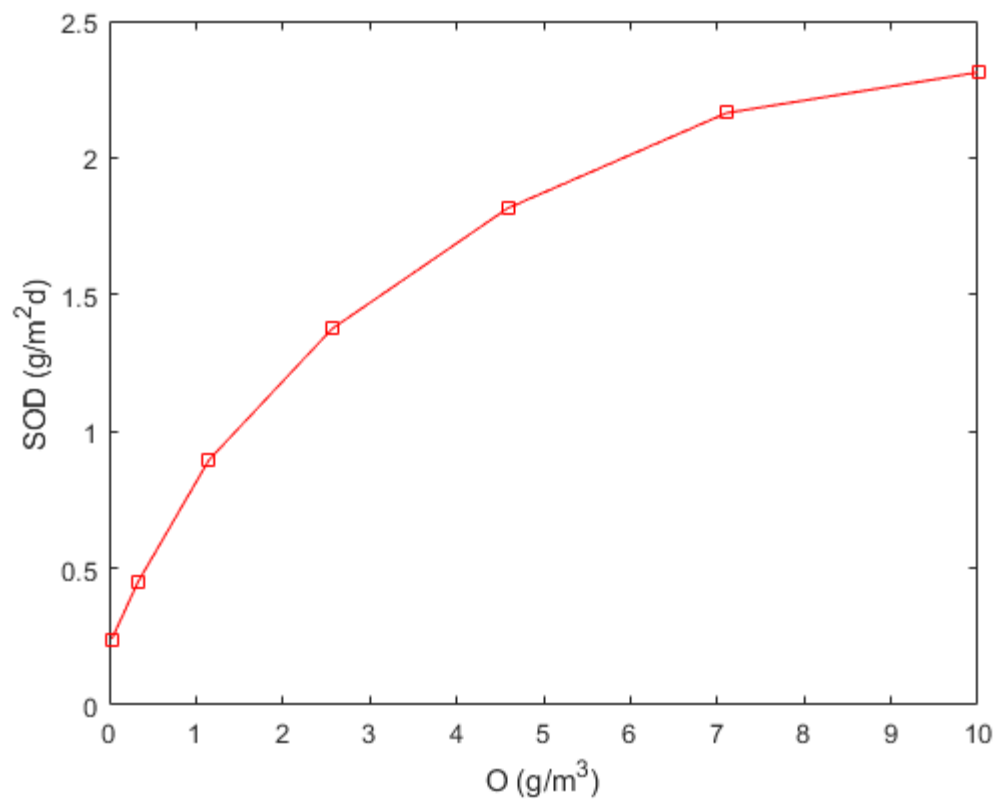
```

function ass3_6
H=0.1;
t=[0,0.125,0.25,0.375,0.5,0.625,0.75];
O=[10,7.11,4.59,2.57,1.15,0.33,0.03];
SOD=-H*tabular_derivative(t,O);
figure(1);
plot(t,SOD,'-go');
xlabel('time (d)');
ylabel('SOD (g/m^2d)');
figure(2);
plot(O,SOD,'-rs');
xlabel('O (g/m^3)');
ylabel('SOD (g/m^2d)');
return

% calculates derivatives at the same points supplied
function [dydx] = tabular_derivative(x,y)
n=length(x); dydx=zeros(n,1);
% use central for interior points
for i=2:n-1
    dydx(i)=(y(i+1)-y(i-1))/(x(i+1)-x(i-1));
end
% use forward for first point
dydx(1) = (y(2)-y(1))/(x(2)-x(1));
% use backward for last point
dydx(n)=(y(n)-y(n-1))/(x(n)-x(n-1));
return

```





```
% well problem: assignment 3: problem 7
function ass3_7_updated
xi=-20.5:1:20.5; eps=1;
yi=xi;
K=1;
[x,y]=meshgrid(xi,yi);
head=hxy(x,y);
v = -50:10:50;
[C,h]=contour(x,y,head,v,'k');
hold on;
[Vx,Vy]=gradient(-K*head); % matlab's gradient function
% [Vx,Vy]=my_gradient_2d(@hxy,x,y,eps); % our equivalent gradient
function
quiver(x,y,Vx,Vy);
hold off;
figure(2);
phi=phi_xy(x,y);
psi=psi_xy(x,y);
v = -40:10:40;
[C,h]=contour(x,y,phi,v,'b');
hold on;
[C,h]=contour(x,y,psi,v,'r');
hold off;
return

% head function
function [hxy]= hxy(x,y)
hw0=0; K=1; a=10; q=100;
f1=(x-a).^2+y.^2;
f2=(x+a).^2+y.^2;
% [i,j] = find(f1 == 0); f1(i,j)=1e-9;
% [i,j] = find(f2 == 0); f2(i,j)=1e-9;
f=f1./f2;
fxy=log(f);
hxy=hw0+q*fxy/(4*pi*K);
return

% phi function
function [phi]= phi_xy(x,y)
a=10; q=100;
f1=(x-a).^2+y.^2;
f2=(x+a).^2+y.^2;
% [i,j] = find(f1 == 0); f1(i,j)=1e-9;
% [i,j] = find(f2 == 0); f2(i,j)=1e-9;
f=f1./f2;
fxy=log(f);
phi=q*fxy/(4*pi);
return

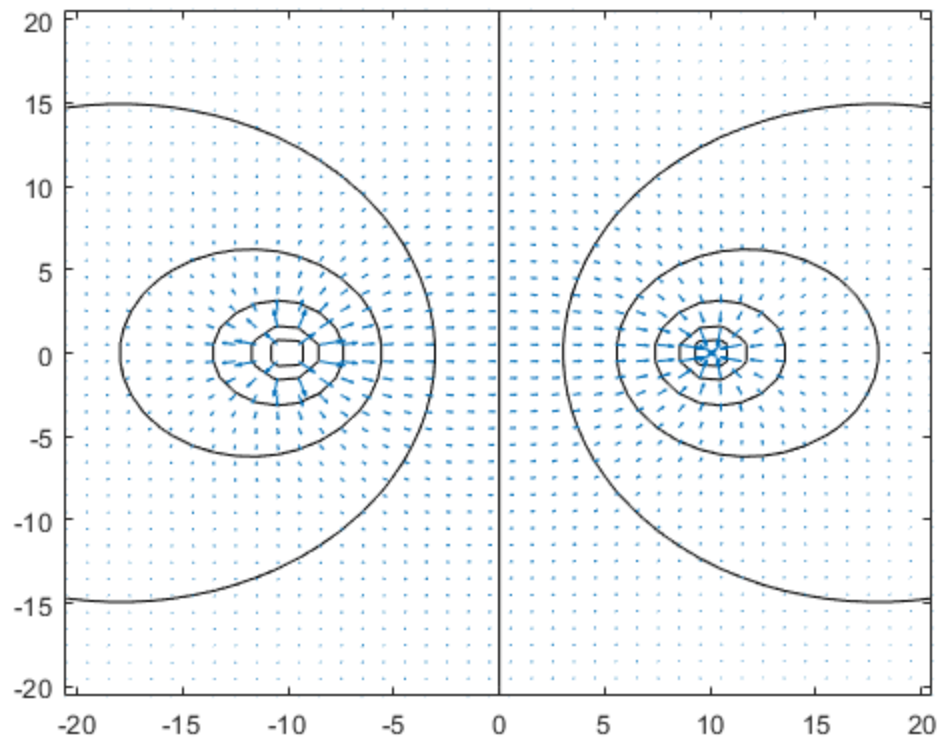
% psi function
function [psi]= psi_xy(x,y)
a=10; q=100;
```

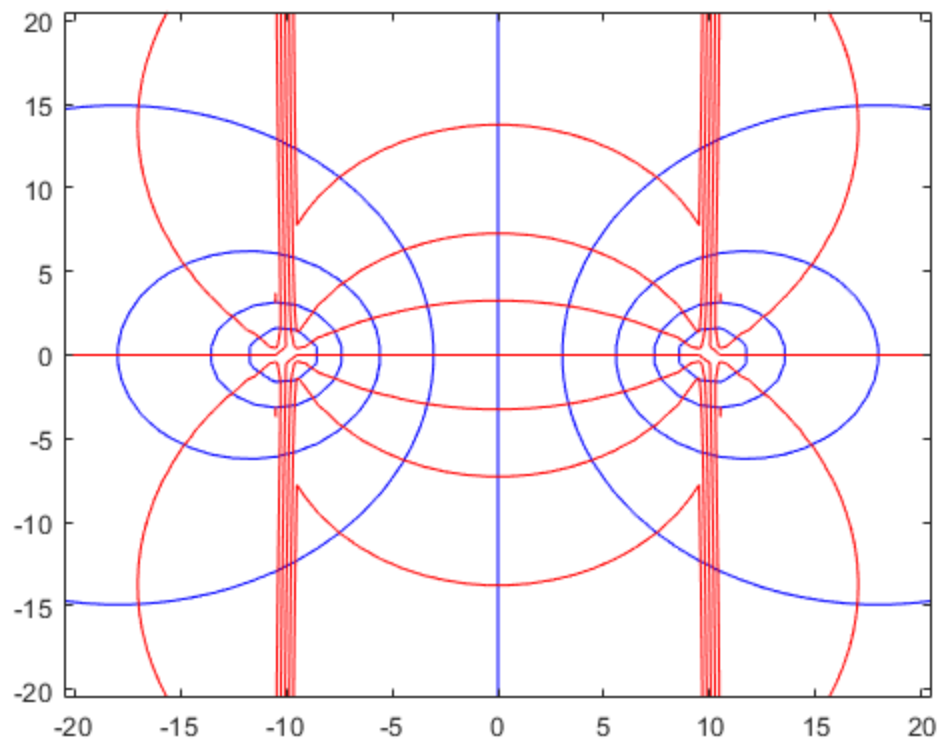
```

fxy=atan(y./(x+a))-atan(y./(x-a));
psi=q*fxy/(2*pi);
return

function [gx,gy] = my_gradient_2d(f,x,y,h)
[m,n]=size(x); K=1;
for i=1:m
    for j=1:n
        gx(i,j)=-K*(feval(f,x(i,j)+h,y(i,j))-feval(f,x(i,j)-
h,y(i,j)))/(2*h);
        gy(i,j)=-K*(feval(f,x(i,j),y(i,j)+h)-feval(f,x(i,j),y(i,j)-
h))/(2*h);
    end
end
return

```





Published with MATLAB® R2020a